

Lucas Augusto da Silva Cardoso (202200332300)

DCA3703 | Programação Paralela | Relato de experimento

TAREFA 9: ESTUDO SOBRE CONCORRÊNCIA E REGIÕES CRÍTICAS EM LISTAS ENCADEADAS COMPARTILHADAS

Este documento detalha a abordagem metodológica e a solução implementada para o Experimento Computacional proposto na disciplina de Programação Paralela, em 04/09/2026.

1. EXPERIMENTO

Escreva um programa que cria tarefas para realizar N inserções em duas listas encadeadas, cada uma associada a uma thread. Cada tarefa deve escolher aleatoriamente em qual lista inserir um número. Garanta a integridade das listas evitando condição de corrida e, sempre que possível, use regiões críticas nomeadas para que a inserção em uma lista não bloqueie a outra. Em seguida, generalize o programa para um número de listas definido pelo usuário. Explique por que, nesse caso, regiões críticas nomeadas não são suficientes e por que o uso de locks explícitos se torna necessário.

2. CONCEITOS RELEVANTES

- **#pragma omp critical (nome):** cria uma região crítica nomeada. Ela garante que apenas uma thread por vez execute o bloco de código associado a aquele nome específico.
- **omp_lock_t lock**: Declara uma variável de do tipo "lock" (um mutex simples fornecido pelo OpenMP). Ela armazena o estado do bloqueio (livre ou ocupado).

- **omp_init_lock(&lock):** Inicializa o lock. Deixa o bloqueio pronto para uso e no estado desbloqueado/livre. Deve ser chamado apenas uma vez antes de qualquer thread tentar usá-lo.
- **omp_set_lock(&lock):** Solicita e obtém a posse do lock (bloqueia o recurso). Se outra thread já tiver a posse, a thread atual fica bloqueada/esperando (espera ocupada) até que o lock seja liberado.
- **omp_unset_lock(&lock):** Libera a posse do lock (desbloqueia o recurso), permitindo que qualquer outra thread que esteja esperando possa adquiri-lo.
- **omp_destroy_lock(&lock)**: Destrói o lock e libera qualquer recurso interno associado a ele. Deve ser chamado quando o lock não for mais necessário.

3. MÁQUINA UTILIZADA

Para a execução deste experimento, estou utilizando uma máquina equipada com um processador AMD Ryzen 7 5700X (8 núcleos físicos e 16 threads), que é o componente central para a avaliação das diretivas OpenMP. O sistema dispõe também de 32 GB de memória RAM e um SSD de 1TB.

4. CÓDIGO

Para a realização deste experimento, foram desenvolvidas três variações do programa. Na primeira versão, utilizou-se regiões críticas nomeadas (#pragma omp critical(nome)). Essa abordagem mostrou-se viável por se tratar de um cenário estático, no qual o número de listas é fixo e conhecido em tempo de compilação. A segunda versão foi implementada sem nenhum mecanismo de sincronização (sem *critical* ou *locks*), com o objetivo de observar experimentalmente a ocorrência de condições de corrida (*race conditions*) e a corrupção de dados na estrutura encadeada. Por fim, na terceira versão, generalizou-se o problema para N listas através do uso de travas explícitas (omp_lock_t). Nessa abordagem, foi possível realizar o bloqueio e desbloqueio dinâmico e individual de cada lista em tempo de execução, garantindo a integridade dos dados de forma escalável.

Na versão 3, para sincronizar o acesso ao vetor dinâmico de listas, utiliza-se um vetor correspondente de travas (omp_lock_t), inicializado via “omp_init_lock”. Ao aplicar “omp_set_lock” e omp_unset_lock diretamente sobre a posição

sorteada (travas[escolha]), estabelece-se um bloqueio de granularidade fina (*fine-grained*): a "thread" protege exclusivamente a lista que está alterando no momento, permitindo que inserções em listas distintas ocorram simultaneamente sem contenção e mantendo a integridade das estruturas encadeadas.

Figura 1: Versão 1 (usando critical nomeado)

```c
#include <omp.h>
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

typedef struct No {
    int valor;
    struct No *proximo;
} No;

static No *criar_no(int valor) {
    No *novo_no = malloc(sizeof(*novo_no));

    if (novo_no != NULL) {
        novo_no->valor = valor;
        novo_no->proximo = NULL;
    }

    return novo_no;
}

static void liberar_lista(No *lista) {
    while (lista != NULL) {
        No *proximo = lista->proximo;
        free(lista);
        lista = proximo;
    }
}

static size_t tamanho_lista(const No *lista) {
    size_t tamanho = 0;

    while (lista != NULL) {
```

```
        tamanho++;
        lista = lista->proximo;
    }

    return tamanho;
}

static void imprimir_lista(const char *nome, const No *lista) {
    printf("%s (%zu elementos):", nome, tamanho_lista(lista));

    while (lista != NULL) {
        printf(" %d", lista->valor);
        lista = lista->proximo;
    }

    putchar('\n');
}

int main(int argc, char **argv) {
    char *fim;
    long n;
    No *lista1 = NULL;
    No *lista2 = NULL;

    if (argc != 2) {
        fprintf(stderr, "Uso: %s N\n", argv[0]);
        return EXIT_FAILURE;
    }

    n = strtol(argv[1], &fim, 10);
    if (*argv[1] == '\0' || *fim != '\0' || n < 0) {
        fprintf(stderr, "N deve ser um inteiro nao negativo.\n");
        return EXIT_FAILURE;
    }

    #pragma omp parallel shared(lista1, lista2)
    {
        #pragma omp single // (cada uma associada a uma thread)
        {
            for (long i = 0; i < n; i++) {
```

```
            unsigned int semente = (unsigned int)time(NULL) ^ (unsigned int)(i *
2654435761UL); // Gera uma semente diferente para cada iteração

            #pragma omp task firstprivate(i, semente) shared(lista1, lista2)
            {
                No *novo_no = criar_no((int)i);
                int escolha;

                if (novo_no == NULL) {
                    fprintf(stderr, "Erro ao alocar memoria para uma insercao.\n");
                    exit(EXIT_FAILURE);
                }

                escolha = (int)(rand_r(&semente) % 2);
                if (escolha == 0) {
                    //usando o critical para proteger a lista1
                    #pragma omp critical(lista1)
                    {
                        novo_no->proximo = lista1;
                        lista1 = novo_no;
                    }
                } else {
                    #pragma omp critical(lista2)
                    {
                        novo_no->proximo = lista2;
                        lista2 = novo_no;
                    }
                }
            }
        }

    }
  }

  imprimir_lista("Lista 1", lista1);
  imprimir_lista("Lista 2", lista2);

  liberar_lista(lista1);
  liberar_lista(lista2);
  return EXIT_SUCCESS;
}
```

Figura 2: Versão 2 (sem critical e lock)

```
int main(int argc, char **argv) {
    char *fim;
    long n;
    long numero_listas;
    No **listas;

    if (argc != 3) {
        fprintf(stderr, "Uso: %s N NUMERO_DE_LISTAS\n", argv[0]);
        return EXIT_FAILURE;
    }

    n = strtol(argv[1], &fim, 10);
    if (*argv[1] == '\0' || *fim != '\0' || n < 0) {
        fprintf(stderr, "N deve ser um inteiro nao negativo.\n");
        return EXIT_FAILURE;
    }

    numero_listas = strtol(argv[2], &fim, 10);
    if (*argv[2] == '\0' || *fim != '\0' || numero_listas <= 0) {
        fprintf(stderr, "O numero de listas deve ser um inteiro positivo.\n");
        return EXIT_FAILURE;
    }

    listas = calloc((size_t)numero_listas, sizeof(*listas));
    if (listas == NULL) {
        fprintf(stderr, "Erro ao alocar memoria para as listas.\n");
        free(listas);
        return EXIT_FAILURE;
    }

    #pragma omp parallel shared(listas, numero_listas)
    {
        #pragma omp single
        {
            for (long i = 0; i < n; i++) {
                unsigned int semente = (unsigned int)time(NULL) ^
                    (unsigned int)(i * 2654435761UL);
```

```
            #pragma omp task firstprivate(i, semente) shared(listas, numero_listas)
            {
                No *novo_no = criar_no((int)i);
                long escolha;

                if (novo_no == NULL) {
                    fprintf(stderr, "Erro ao alocar memoria para uma insercao.\n");
                    exit(EXIT_FAILURE);
                }

                escolha = rand_r(&semente) % numero_listas;
                novo_no->proximo = listas[escolha];
                listas[escolha] = novo_no;
            }
        }
    }
}

for (long i = 0; i < numero_listas; i++) {
    imprimir_lista((size_t)(i + 1), listas[i]);
    liberar_lista(listas[i]);
}

free(listas);
return EXIT_SUCCESS;
}
```

Figura 3: Versão 2 (usando locks)

```
int main(int argc, char **argv) {
    char *fim;
    long n;
    long numero_listas;
    No **listas;
    omp_lock_t *travas;

    if (argc != 3) {
        fprintf(stderr, "Uso: %s N NUMERO_DE_LISTAS\n", argv[0]);
        return EXIT_FAILURE;
    }
```

```c
n = strtol(argv[1], &fim, 10);
if (*argv[1] == '\0' || *fim != '\0' || n < 0) {
    fprintf(stderr, "N deve ser um inteiro nao negativo.\n");
    return EXIT_FAILURE;
}

numero_listas = strtol(argv[2], &fim, 10);
if (*argv[2] == '\0' || *fim != '\0' || numero_listas <= 0) {
    fprintf(stderr, "O numero de listas deve ser um inteiro positivo.\n");
    return EXIT_FAILURE;
}

listas = calloc((size_t)numero_listas, sizeof(*listas));
travas = malloc((size_t)numero_listas * sizeof(*travas));
if (listas == NULL || travas == NULL) {
    fprintf(stderr, "Erro ao alocar memoria para as listas.\n");
    free(listas);
    free(travas);
    return EXIT_FAILURE;
}

for (long i = 0; i < numero_listas; i++) {
    omp_init_lock(&travas[i]);
}

#pragma omp parallel shared(listas, travas, numero_listas)
{
    #pragma omp single
    {
        for (long i = 0; i < n; i++) {
            unsigned int semente = (unsigned int)time(NULL) ^
                (unsigned int)(i * 2654435761UL);

            #pragma omp task firstprivate(i, semente) shared(listas, travas, numero_listas)
            {
                No *novo_no = criar_no((int)i);
                long escolha;

                if (novo_no == NULL) {
                    fprintf(stderr, "Erro ao alocar memoria para uma insercao.\n");
                    exit(EXIT_FAILURE);
```

```
            }

            escolha = rand_r(&semente) % numero_listas;
            omp_set_lock(&travas[escolha]);
            novo_no->proximo = listas[escolha];
            listas[escolha] = novo_no;
            omp_unset_lock(&travas[escolha]);
          }
        }
      }
    }

    for (long i = 0; i < numero_listas; i++) {
      imprimir_lista((size_t)(i + 1), listas[i]);
      liberar_lista(listas[i]);
      omp_destroy_lock(&travas[i]);
    }

    free(listas);
    free(travas);
    return EXIT_SUCCESS;
}
```

5. RESULTADOS

A verificação de corretude consistiu em comparar a soma dos elementos de todas as listas com o total N = 100.000 inserções solicitadas, executadas via tarefas OpenMP em ambiente multi-thread (OMP_NUM_THREADS=8).

Quadro 1: Análise dos códigos

Versão	Estrutura	Mecanismo de Proteção	Total Obtido	Comportamento e Corretude
Versão 1	2 listas fixas	critical(lista1) e critical(lista2)	100.000	Correto. Mecanismo viável apenas por ter número estático de listas conhecido em tempo de

				compilação.
Versão 2	N listas dinâmicas	Nenhum (Acesso direto)	99.997	Incorreto. Ocorrência de lost update (condição de corrida não determinística) devido a acessos concorrentes simultâneos.
Versão 3	N listas dinâmicas	Locks explícitos (omp_lock_t por lista)	100.000	Correto. Bloqueio dinâmico individual (fine-grained) que garante integridade e escala com o número de listas.

A integridade das listas encadeadas foi validada contando o total de elementos ao final do processamento de N = 100.000 inserções em 8 “threads” (“OMP_NUM_THREADS=8”). As “Versões 1” (duas listas com *critical* nomeado) e “3” (quatro listas com “locks” explícitos) preservaram todos os dados, atingindo exatamente 100.000 elementos. Por outro lado, a “Versão 2” (quatro listas sem sincronização) registrou apenas 99.997 elementos, comprovando a corrupção da memória por acessos simultâneos.

A perda de dados na Versão 2 decorre do fenômeno de atualização perdida (“lost update”). Como a inserção no cabeçalho não é uma operação atômica, duas “tasks” que tentam modificar a mesma lista simultaneamente leem o mesmo nó inicial. A “thread” que finaliza por último sobrescreve a alteração da primeira, tornando o nó anterior inalcançável. O resultado não determinístico de 3 nós perdidos ilustra a gravidade dessa condição de corrida.

A Versão 1 evitou esse problema usando regiões críticas nomeadas independentes (“critical(lista1)” e “critical(lista2)”), permitindo que duas listas fossem alteradas em paralelo. Contudo, essa técnica exige nomes fixos no código-fonte em tempo de compilação. Usar um único “critical” global para contornar isso em N listas serializaria o programa, pois qualquer inserção bloquearia o acesso a todas as demais listas.

Por fim, a Versão 3 garantiu escalabilidade ao parear um vetor dinâmico de listas com um vetor de travas (“omp_lock_t”). O uso de “omp_set_lock(&travas[escolha])” estabelece um controle de “granularidade fina (“fine-grained”)”: apenas a lista sorteada é bloqueada. Isso permite que “tasks” destinadas a listas diferentes executem em paralelo sem contenção, unindo corretude total e alto desempenho para N listas.

6. CONSIDERAÇÕES FINAIS

O experimento evidenciou como a escolha das primitivas de sincronização no OpenMP impacta diretamente a corretude e o paralelismo em estruturas de dados compartilhadas. A “Versão 2” (sem sincronização) confirmou que acessos concorrentes desprotegidos causam condições de corrida e corrupção de memória por atualizações perdidas (lost updates).

Embora a “Versão 1” (regiões críticas nomeadas) tenha garantido a integridade dos dados, sua aplicação fica restrita a cenários estáticos, pois exige que os nomes das regiões sejam definidos em tempo de compilação. Já a “Versão 3“ provou ser a solução ideal para a generalização em N listas. O uso de um vetor dinâmico de travas explícitas (omp_lock_t) estabeleceu um controle de granularidade fina, assegurando a integridade das listas sem gerar gargalos globais e permitindo a execução paralela eficiente para qualquer quantidade de listas.